

Helix Thready — ERD Regeneration (er-export)

Table of Contents

- 1. Regeneration pipeline (diagram)
- 2. Inputs: the schema source of truth
- 3. Tools (choose one)
- 4. Step-by-step
- 5. Drift gate (regenerated vs committed)
- 6. Provenance & open items

Rev	Date	Author	Change
1	2026-07-22	swarm (database, materials)	Initial: how to regenerate the ERD from the schema (pipeline + tools + drift gate)

How to regenerate the entity-relationship diagrams in `../erd.md` from the schema — so the ERD is a **derived artifact** kept in lock-step with the migrations, not a hand-drawing that silently drifts. The canonical `erd.md` diagrams are currently hand-authored Mermaid `[VERIFIED — see ../erd.md]`; this document specifies the repeatable pipeline that regenerates and **verifies** them against the live applied schema.

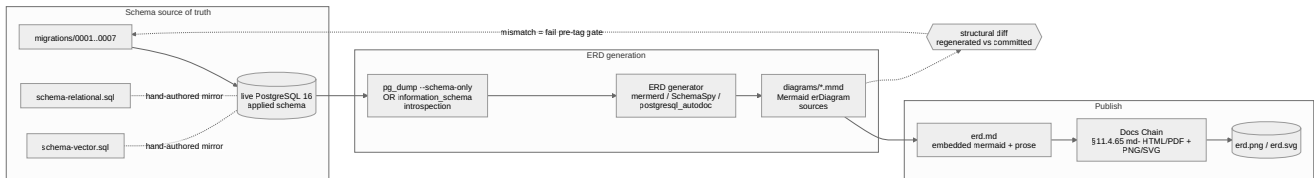
Table of Contents

- 1. Regeneration pipeline (diagram)
- 2. Inputs: the schema source of truth
- 3. Tools (choose one)
- 4. Step-by-step
- 5. Drift gate (regenerated vs committed)
- 6. Provenance & open items

1. Regeneration pipeline (diagram)

Source: `diagrams/er-export-pipeline.mmd`.

Rendered PNG/SVG exported via Docs Chain (\$11.4.65).



Explanation (for readers/models that cannot see the diagram). The pipeline reads left-to-right in three stages. On the left, the **schema source of truth** has one authoritative producer and two mirrors: the ordered `migrations/0001..0007` are what is actually `Apply`-ed to produce the **live PostgreSQL 16** applied schema (the solid `MIG → PG` edge), while `schema-relational.sql` and `schema-vector.sql` are hand-authored **mirrors** of that same shape (the dashed edges) kept for review — the migrations, not the mirror files, are what the database ends up matching. Regeneration therefore always targets the **live applied schema**, because that is the ground truth that the ERD must describe.

The middle **ERD generation** stage extracts the schema and renders diagrams: `pg_dump --schema-only` (or an `information_schema` / `pg_catalog` introspection query) captures the tables, columns, keys and foreign keys, an **ERD generator** (mermerd, SchemaSpy, or `postgresql_autodoc`) turns that into Mermaid `erDiagram` text, and the result is written to the `diagrams/*.mmd` sibling files. The right **publish** stage embeds those `.mmd` sources into `erd.md` as fenced ``mermaid blocks` (each with its mandatory prose paragraph per `[CONVENTIONS §4](../CONVENTIONS.md)`), and `**Docs Chain**` (`$11.4.65`) exports the whole document to HTML/PDF and the diagrams to `erd.png / erd.svg`.

The dotted **drift gate** node closes the loop: the freshly regenerated `.mmd` is structurally diffed against the committed one, and a mismatch **fails the pre-tag gate** and points back at the migrations — meaning a schema change shipped without regenerating the ERD is caught before a release tag, exactly as the migration structural-diff gate works (`migration-strategy.md §10`).

2. Inputs: the schema source of truth

The ERD is generated from the **applied schema**, obtained by running the migrations into a throwaway database (never from the hand-authored `.sql` mirror directly — that is what the drift gate protects against):

```
# 1. Stand up an ephemeral Postgres and apply the canonical chain + seed lookups.
#   (dev parity: the same migrations run on SQLite, but the ERD is generated from PG 16
#   because pgvector types + partitioned parents must be introspected – see $6.)
docker run --rm -d --name thready_erd -e POSTGRES_PASSWORD=erd -p 5433:5432 pgvector/
pgvector:pg16
export DSN="postgres://postgres:erd@localhost:5433/postgres?sslmode=disable"

# Apply migrations in version order. Feed each file's Up section (the runner would; for a
# raw psql preview we extract the block between the '-- +thready Up'/'-- +thready Down'
# markers so psql never also runs the Down half):
up() { awk '/^-- \+thready Up/{u=1;next} /^-- \+thready Down/{u=0} u' "$1"; }
for f in 0001_init 0002_classification 0003_assets 0004_billing \
        0005_events_audit 0006_vector_collections; do
  up "$f" | psql "$DSN" -v ON_ERROR_STOP=1
done
# 0007 (CONCURRENTLY) is applied outside a transaction; for a schema dump the indexes are
# not required – ERDs describe tables/keys/FKs, not secondary indexes.
psql "$DSN" -v ON_ERROR_STOP=1 -f ./seed.sql # optional: lookups for a populated preview
```

The `-- +thready Up/Down` markers make each file loadable by the Go runner; for a raw `psql -f` preview strip the `-- +thready Down` half (everything after that marker), since `psql` would otherwise also execute the Down. In practice, generate from a DB migrated by the service's own migrate step so Up/Down handling is exactly the runtime path.

3. Tools (choose one)

[DEFAULT – adjustable] — the exact generator is not mandated; any of these produces the Mermaid the pipeline needs. The in-house **Docs Chain** owns the final md→HTML/PDF+image export [CONSTITUTION §11.4.65] [IN-HOUSE: docs_chain]; the ERD-text generator feeding it is the adjustable part.

Tool	Emits	Notes
mermerd	Mermaid <code>erDiagram</code> directly	Best fit — output pastes straight into <code>diagrams/*.mmd</code> ; connects to <code>\$DSN</code> , can select table subsets for the 5 domain ERDs.
SchemaSpy	HTML + relationship graphs	Rich HTML; convert its relationships to Mermaid or keep as a supplementary browse view.
postgresql_autodoc	Dot / HTML / dia	Classic; post-process Dot → Mermaid.
pg_dump --schema-only + hand map	raw DDL	Fallback when no generator is available; script the DDL→erDiagram mapping.

Partition-aware / pgvector caveat: generators that introspect `pg_catalog` see partitioned **parents** and their partitions as separate relations, and see `vector(N)` columns as a user type. Filter child partitions (`*_YYYY_MM`, `*_default`) and render only the parent, and keep the `vectordb_*` collections in the vector sub-diagram — matching how `erd.md` §4/§7 present them.

4. Step-by-step

```
# (after §2 produced a migrated DB at $DSN)

# a. Generate the master ERD (all base tables) as Mermaid.
mermerd --connectionString "$DSN" --schema public --useAllTables \
  --encloseWithMermaidBackticks=false > /tmp/erd-overview.mmd

# b. Generate each DOMAIN ERD by selecting its tables (mirrors erd.md §3-§6).
mermerd --connectionString "$DSN" --selectedTables
  accounts,users,roles,permissions,role_permissions,memberships \
  > /tmp/erd-tenancy.mmd
mermerd --connectionString "$DSN" --selectedTables
  messengers,messenger_accounts,channels,threads,posts,replies,hashtags,categories,post_hashtags,reply_
  \
  > /tmp/erd-ingestion.mmd
mermerd --connectionString "$DSN" --selectedTables
  processing_state,skills,skill_runs,generated_artifacts,assets,asset_links \
  > /tmp/erd-processing-assets.mmd
mermerd --connectionString "$DSN" --selectedTables
  events,event_subscriptions,plans,subscriptions,usage_records,invoices,audit_log \
  > /tmp/erd-billing-audit.mmd

# c. Review the generated .mmd, filter child partitions, then update the siblings in
#   ../diagrams/ and the embedded ``mermaid blocks in ../erd.md. Each embedded diagram
#   MUST keep its "Explanation (for readers/models...)" prose paragraph (CONVENTIONS §4).

# d. Export images + HTML/PDF via Docs Chain (the authoritative renderer, §11.4.65).
docs_chain export --input ../erd.md --emit png,svg,html,pdf # [IN-HOUSE: docs_chain]
```

Manual step (kept honest): the **prose explanation** under each diagram and the soft-vs-hard-FK annotations are authored by a human/model — a generator emits topology, not the multi-paragraph narrative CONVENTIONS §4 mandates. Regeneration refreshes the *topology*; the prose is reviewed and updated in the same change.

5. Drift gate (regenerated vs committed)

Because there is **no server-side CI** [CONSTITUTION §11.4.156], the ERD-vs-schema check runs in the local pre-tag full-suite retest [§11.4.40], alongside the migration structural-diff already specified in migration-strategy.md §10:

```
# Regenerate into a temp file and diff against the committed sibling; non-empty = drift.
mermerd --connectionString "$DSN" --useAllTables > /tmp/erd-overview.new.mmd
if ! diff -q <(normalize /tmp/erd-overview.new.mmd) <(normalize ../diagrams/erd-overview.mmd);
then
    echo "ERD DRIFT: schema changed but diagrams/erd-overview.mmd was not regenerated" >&2
    exit 1 # blocks the <PREFIX>-<ver> tag until the ERD is regenerated + prose updated
fi
```

`normalize` strips ordering/whitespace noise so only **structural** changes (a new table, column, or FK) trip the gate. This makes the ERD a *tested* derived artifact: a migration that adds `usage_rollups` or `billing_events` (this pack's `0003_billing_metering.sql`) cannot be tagged until the ERD reflects it.

6. Provenance & open items

- **VERIFIED:** the ERD in `../erd.md` is currently hand-authored Mermaid with sibling `.mmd` files in `../diagrams/`; Docs Chain is the mandated md→HTML/PDF renderer [CONSTITUTION §11.4.65] [IN-HOUSE: docs_chain]; the migrations are the true producer of the applied schema (migration-strategy.md).
- **ASSUMED / [DEFAULT — adjustable]:** the specific ERD-text generator (mermerd / SchemaSpy / autodoc), the `pgvector/pgvector:pg16` preview image, and the `normalize` filter details — these are tooling choices, not schema, and can change without a migration.
- **[OPEN: erd-autogen-adoption]** The pipeline is specified and runnable but adopting an auto-generator to *replace* the hand-authored diagrams (vs. only *checking* them) is a follow-up: the domain sub-diagrams and their prose are curated for readability, so the near-term use is the **drift gate** (§5), with full autogen tracked as a later refinement. This is honest status — the check is real; full generation is not yet the source of the committed diagrams.

Made with love ♥ by Helix Development.